

Lectures on Monte Carlo Theory

Chapter 2: The Theory of Generators

Based on Leskeä & Vihola

Chapter 2 Outline

- 1 2.1 Introduction
- 2 2.2 Random Permutations
- 3 2.3 Pseudorandom Number Generators in Python
- 4 2.4 Simple Generators: Historical Perspective
- 5 2.5 Testing Good Properties of PRNGs
- 6 2.6 Bibliographical Notes

2.1 Introduction: Why Do We Need Random Numbers on a Computer? I

Randomness is a cornerstone of Monte Carlo methods. Any algorithm that relies on randomness requires a source of *independent, identically distributed uniform random numbers*. Such numbers are needed for simulating stochastic processes, estimating integrals by sampling, generating cryptographic keys, and performing statistical resampling.

A digital computer, however, is a *deterministic machine*: given the same input, it always produces the same output. It therefore cannot produce genuinely random numbers. Instead, a computer runs a carefully designed deterministic algorithm that produces a sequence whose *statistical properties* closely mimic those of a truly random sequence. Such an algorithm is called a **pseudorandom number generator (PRNG)**. The word “pseudo” (from the Greek for “false”) reminds us that the numbers are not truly random, but they are designed to appear random to any practical test.

A motivating failure: the Middle-Square Method (von Neumann, 1940s). The idea is deceptively simple: start with a four-digit integer (the “seed”), square it, keep the four middle digits as the next number, and repeat. For example, starting from 43:

$$43^2 = 1849, \quad 84^2 = 7056, \quad 05^2 = 0025, \quad 02^2 = 0004, \quad 00^2 = 0000 \rightarrow 0000 \rightarrow 0000 \rightarrow \dots$$

2.1 Introduction: Why Do We Need Random Numbers on a Computer? II

The sequence collapses to the constant value 0000 after just five steps and stays there forever. This illustrates the fundamental difficulty: any deterministic generator operating on a *finite* set of states must eventually revisit a state and then cycle. Without careful design the cycle can be embarrassingly short, rendering the sequence useless for any application needing more than a handful of values.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? III

Definition 2.1.1 — Pseudorandom Number Generator (PRNG)

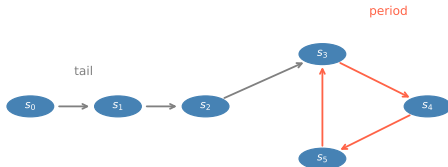
A PRNG is a 5-tuple (S, s_0, f, V, g) where:

- S is a finite **state space** (the set of all possible internal states of the generator),
- $s_0 \in S$ is the **initial state**, also called the *seed* (the starting point; changing the seed produces a different sequence),
- $f : S \rightarrow S$ is the **transition function**: the internal state evolves one step at a time as $s_{n+1} = f(s_n)$,
- V is a finite set of **output values** (e.g., integers in $\{0, \dots, 2^{32} - 1\}$),
- $g : S \rightarrow V$ is the **output function**: the i -th output value is $u_i = g(s_i)$, for $i = 0, 1, 2, \dots$

Because S is finite, the sequence of states $(s_n)_{n \geq 0}$ must eventually revisit a previous state and then repeat from that point forward. The length of the repeating cycle is called the **period** of the generator. The period cannot exceed $|S|$, the total number of states. Longer periods are desirable because the sequence takes more steps before repeating, reducing the risk of using the same “random” values twice.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? IV

PRNG state sequence: tail + periodic cycle



The figure above illustrates the structure schematically: after a “tail” of states leading into the cycle, the generator repeats indefinitely. A well-designed generator has a period close to $|S|$ and a tail that is as short as possible.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? V

Key Insight: Period is Everything

For a simulation requiring n samples, the period must satisfy $period \gg n$; otherwise the sequence simply repeats within the simulation and introduces spurious correlations. For modern simulations with $n \sim 10^9$ samples, a period of 2^{128} (as in PCG64) is essentially inexhaustible: at 10^9 samples per second it would take 10^{19} times the age of the universe to cycle through.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? VI

Definition 2.1.1.1 — Three Equivalent Types of Random Sequences

(i) Sequence of random numbers from $[0, 1)$. We say $U_1, U_2, \dots$ is a sequence of random numbers from the unit interval $[0, 1)$ if each U_j is a random variable with the *uniform distribution* on $[0, 1)$:

$$\mathbb{P}(U_j \leq t) = \begin{cases} 0 & t < 0, \\ t & 0 \leq t < 1, \\ 1 & t \geq 1, \end{cases}$$

and the variables are *independent and identically distributed (i.i.d.)*:

$$\mathbb{P}(U_1 \leq t_1, \dots, U_n \leq t_n) = t_1 t_2 \cdots t_n, \quad 0 \leq t_i \leq 1.$$

We write $U \sim \mathcal{U}[0, 1)$ to denote this distribution.

(ii) Sequence of random bits. The B_j are i.i.d. taking values in $\{0, 1\}$ with $\mathbb{P}(B_j = 0) = \mathbb{P}(B_j = 1) = \frac{1}{2}$, so any particular bit string of length n has probability $1/2^n$.

(iii) Sequence of random integers from $\bar{M}$. The Y_j are i.i.d. uniformly distributed on $\{0, 1, \dots, M-1\}$ where $M \geq 2$ is a fixed integer. We write $Y \sim \mathcal{U}(\bar{M})$.

These three representations are *interconvertible*. Given $u_i \in [0, 1)$, the integer $y_i = \lfloor M u_i \rfloor$ lies in

2.2 Random Permutations: Shuffling and its Applications I

A **permutation** of the set $\{1, \dots, n\}$ is an arrangement of its elements in some order. For example, with $n = 3$ the possible permutations are $(1, 2, 3)$, $(1, 3, 2)$, $(2, 1, 3)$, $(2, 3, 1)$, $(3, 1, 2)$, $(3, 2, 1)$ — six arrangements in total ($3! = 6$).

A **random permutation** is one chosen uniformly at random from all $n!$ possible arrangements, so each arrangement has probability $1/n!$. Generating random permutations is a fundamental subroutine in Monte Carlo algorithms: it underlies random shuffling of data, randomised experiments (assigning treatments to subjects), bootstrapping, and permutation-based statistical tests.

The naive approach and why it fails. One might try picking elements one at a time uniformly at random from $\{1, \dots, n\}$, discarding repeats until all n elements have been chosen. This is slow: the expected number of draws required diverges logarithmically because later draws must avoid an ever-growing list of already-chosen elements (the coupon-collector problem). The *Fisher-Yates shuffle* avoids this entirely by construction.

2.2 Random Permutations: Shuffling and its Applications II

Algorithm 1: Fisher-Yates Shuffle (also called the Knuth Shuffle)

Require: An integer n (the number of elements); pseudorandom numbers $\mathbf{u} = (u_1, \dots, u_n)$ with each $u_i \in [0, 1)$.

Ensure: A uniformly random permutation of $\{1, \dots, n\}$.

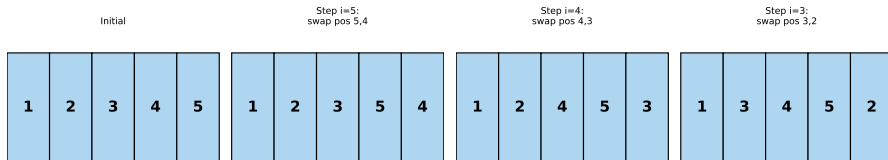
- 1 Initialise array: $S[i] \leftarrow i$ for $i = 1, \dots, n$.
- 2 **for** $i = n$ **downto** 1 **do**:
- 3 $k \leftarrow 1 + \lfloor i u_i \rfloor$ (this selects a uniformly random index in $\{1, \dots, i\}$)
- 4 **Swap**($S[k], S[i]$)
- 5 **end for**
- 6 **return** S

The key idea is elegant: at step i , we choose one of the i elements in positions $1, \dots, i$ uniformly at random and move it into position i . Once placed, an element is never touched again. After n steps, every element has been placed exactly once, and by induction every ordering is equally likely with

2.2 Random Permutations: Shuffling and its Applications III

probability $\frac{1}{n} \cdot \frac{1}{n-1} \cdots \frac{1}{1} = \frac{1}{n!}$. The algorithm runs in $\mathcal{O}(n)$ time (exactly one swap per step), which is optimal.

Fisher-Yates Algorithm: Shuffling [1, 2, 3, 4, 5]



2.2 Random Permutations: Shuffling and its Applications IV

Correctness of Fisher-Yates: a step-by-step argument. We prove by induction that after the loop has processed positions $n, n-1, \dots, i+1$, the elements now in positions $i+1, i+2, \dots, n$ form a uniformly random arrangement of the $n-i$ elements assigned to them.

Base case ($i = n$): We pick position k uniformly from $\{1, \dots, n\}$ and swap $S[k]$ with $S[n]$. Thus the element in position n is uniform over $\{1, \dots, n\}$.

Inductive step: Assume the elements in positions $i+1, \dots, n$ are already a uniform random arrangement of their $n-i$ elements. At step i , the i elements remaining in positions $1, \dots, i$ are equally likely to be any i -element subset (because all $n!$ global orderings are still reachable). We pick one of these i elements uniformly and move it to position i . This maintains the invariant, and after step 1 every arrangement of $\{1, \dots, n\}$ occurs with probability $1/n!$.

2.2 Random Permutations: Shuffling and its Applications V

Algorithm 2: Perm-Sort — Generating a Random Permutation by Sorting

An alternative method assigns a uniform random “key” to each element and then sorts by key:

- 1 Initialise: $S[i] \leftarrow i$ for $i = 1, \dots, n$.
- 2 Generate $u_1, \dots, u_n$ i.i.d. from $\mathcal{U}[0, 1)$.
- 3 Sort S so that $u_{S[1]} \leq u_{S[2]} \leq \dots \leq u_{S[n]}$.
- 4 **return** S .

Both algorithms produce a uniform random permutation given a truly uniform input. Fisher-Yates is preferred in practice because it runs in $\mathcal{O}(n)$ time, while Perm-Sort requires $\mathcal{O}(n \log n)$ time for the sorting step. However, Perm-Sort is sometimes easier to parallelise because the sort can be distributed across processors.

2.2 Random Permutations: Shuffling and its Applications VI

2.2.2 Shuffling a Deck of Cards: How Many Shuffles Are Enough?

A standard deck of $n = 52$ cards can be identified with $\{1, \dots, 52\}$. There are $52! \approx 8.07 \times 10^{67}$ possible orderings — a number so large that if you shuffled a deck every second since the Big Bang, you would still not come close to exploring all arrangements.

A *card-shuffling scheme* is a Markov chain $(X_k)_{k \geq 0}$ on the symmetric group $\mathcal{S}_{52}$ (the set of all 52! permutations) whose stationary distribution is the uniform distribution. The key question is: how many steps are needed to reach approximate uniformity?

The distance from uniformity after k steps is measured by the **total variation distance** d_k :

$$d_k = \frac{1}{2} \sum_{\sigma \in \mathcal{S}_n} \left| \mathbb{P}(X_k = \sigma) - \frac{1}{n!} \right|.$$

Here $d_k = 1$ (maximum) when the current arrangement is deterministic (no randomness at all), and $d_k = 0$ (minimum) when the deck is perfectly uniformly distributed. As more shuffles are applied, $d_k \rightarrow 0$.

Common shuffling methods and their properties:

2.2 Random Permutations: Shuffling and its Applications VII

Top-To-Random: at each step, remove the top card and insert it into a uniformly random position among the n available slots. This is the simplest scheme to analyse theoretically.

Riffle Shuffle (Gilbert-Shannon-Reeds model): cut the deck into two halves, where the cut position follows a binomial distribution $\text{Bin}(n, 1/2)$; then interleave by repeatedly dropping a card from whichever pile is larger (with probability proportional to pile size). This is the shuffle most people perform physically when playing cards.

Time-Reversed Riffle Shuffle: the time-reversed operation interleaves two packs by independently inserting each card of one pack into the other; it is useful theoretically because its transitions are easier to compute.

Key Result: The Seven-Shuffle Theorem (Bayer and Diaconis, 1992)

For a standard deck of $n = 52$ cards, approximately **7 riffle shuffles** are needed before the total variation distance d_k drops below $\frac{1}{2}$. With 6 shuffles the deck is statistically far from uniform; with 8 it is essentially perfectly mixed. This surprising “cut-off phenomenon” — where the chain abruptly transitions from “not mixed” to “mixed” around a critical number of steps — was a landmark result in the theory of Markov chain mixing times.

2.3 Pseudorandom Number Generators in Python I

Modern scientific computing in Python uses the `numpy.random` module, which provides a family of high-quality PRNGs through a clean API. The recommended entry point is `numpy.random.default_rng(seed)`, which returns a Generator object backed by the **PCG64** algorithm.

2.3 Pseudorandom Number Generators in Python II

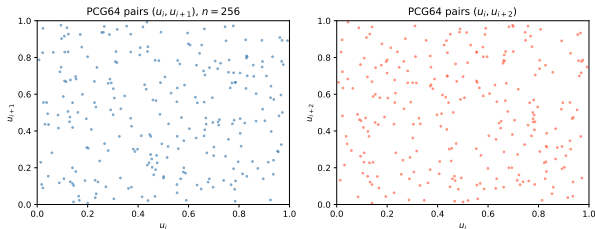
2.3.1 The NumPy Generator Hierarchy

The NumPy API separates *bit generators* (which produce raw random bits) from *generators* (which transform those bits into useful distributions).

- `default_rng(seed)`: the recommended starting point for all new code; creates a Generator backed by PCG64.
- **PCG64** (Permuted Congruential Generator, 64-bit output): period 2^{128} ; passes all standard statistical test batteries; supports multiple independent streams via the `advance` method, which is essential for parallel simulations.
- **MT19937** (Mersenne Twister): period $2^{19937} - 1$; state space is 624 integers of 32 bits each; historically dominant but fails some TestU01 BigCrush tests and has a large state that makes it slow to initialise; used by Python's built-in `random` module and legacy NumPy (`np.random.seed`).
- **Philox** and **SFC64**: alternative high-quality generators that support multiple independent streams and parallel simulations with provable independence guarantees.

2.3 Pseudorandom Number Generators in Python III

Why PCG64 instead of MT19937? The Mersenne Twister's period of $2^{19937} - 1$ is astronomically large, but its very large state (624 32-bit integers = 2.5 KB) makes it slow to initialise. More importantly, it fails some tests in the TestU01 BigCrush battery and is not cryptographically secure. PCG64's period of 2^{128} is smaller in exponent but is still inexhaustible in practice, and it eliminates MT19937's lattice structure through its nonlinear output permutation (see Section 2.4.1 for why lattice structure is a problem).



2.3 Pseudorandom Number Generators in Python IV

The figure above shows 10,000 consecutive pairs (u_i, u_{i+1}) produced by PCG64. The points fill the unit square densely and uniformly, with no visible structure. This should be contrasted with the lattice patterns visible in the pairs produced by simple LCGs (Section 2.4.1), where all points lie on a small number of parallel lines.

2.3 Pseudorandom Number Generators in Python V

Converting Between Output Types in Practice

Given a uniform number $u_i \in [0, 1)$ from the generator, conversion to an integer $y_i \in \{0, \dots, M - 1\}$ uses the floor function:

$$y_i = \lfloor Mu_i \rfloor.$$

Here $\lfloor \cdot \rfloor$ (floor, pronounced “floor of”) rounds down to the nearest integer. For example, if $u_i = 0.731$ and $M = 6$, then $y_i = \lfloor 6 \times 0.731 \rfloor = \lfloor 4.386 \rfloor = 4$.

To obtain a random bit, one should first convert u_i to an integer y_i with $M = 2^k$, then read the individual bits of y_i . The naive approach of setting $b_i = \mathbf{1}[u_i \geq 0.5]$ is dangerous: it keeps only the single most significant bit of u_i and discards all remaining bits. A generator could produce a sequence that passes this test while having all lower-order bits set to zero — such a sequence would immediately fail any proper bit-level test.

2.3.1 Python Code: Using NumPy PRNGs

```
import numpy as np
from numpy.random import PCG64, MT19937, Generator

# --- Recommended: PCG64 via default_rng -----
rng = np.random.default_rng(seed=42)

u      = rng.random(10)           # 10 uniform numbers in [0, 1)
y      = rng.integers(0, 10, 10)  # 10 integers in {0, ..., 9}
perm   = rng.permutation(52)     # random permutation of {0,...,51}
print("Uniform:", u[:5])
print("Integers:", y[:5])
print("Permutation:", perm[:8], "...")

# --- Explicitly choose bit generator -----
rng_pcg = Generator(PCG64(seed=0))
rng_mt  = Generator(MT19937(seed=0))
print("PCG64: ", rng_pcg.random(5))
print("MT19937:", rng_mt.random(5))

# --- Multiple independent streams (PCG64 advance) -----
# Advance the second stream by 2^64 steps so that the two streams
# are statistically independent even when run in parallel.
bg1 = PCG64(seed=0)
bg2 = PCG64(seed=0)
bg2.advance(2**64)                # jump ahead 2^64 steps
rng1 = Generator(bg1)
rng2 = Generator(bg2)
```

2.3.2 Cryptographic PRNGs and RC4 I

For *cryptographic* applications, passing statistical tests is not sufficient. A cryptographic PRNG (CSPRNG) must be *computationally unpredictable*: even if an adversary observes a long prefix of the output stream, no efficient algorithm should be able to predict the next bit with probability significantly better than $1/2$.

Why statistical testing alone is not enough. A statistical test battery such as TestU01 checks whether the output *looks* uniform and independent. It does *not* prevent an adversary from recovering the internal state from the output and thereby predicting all future values. CSPRNGs close this gap by requiring that state recovery be computationally infeasible under standard mathematical hardness assumptions.

RC4 (Rivest Cipher 4) was designed by Ron Rivest in 1987 and became the workhorse of early web security (SSL/TLS, WEP wireless encryption, PDF encryption) because of its extreme simplicity and speed. RC4 maintains a permutation S of $\{0, \dots, n-1\}$ (with $n = 256$ in standard use) and generates one output byte per step. It consists of two phases:

- **Key-Scheduling Algorithm (KSA)**: initialises the permutation S by mixing in the secret key K .

2.3.2 Cryptographic PRNGs and RC4 II

- **Pseudo-Random Generation Algorithm (PRGA)**: streams output bytes by continuously updating and querying the permutation.

2.3.2 Cryptographic PRNGs and RC4 III

Algorithm 3: RC4 Key-Scheduling Algorithm (KSA)

Require: Integer n (typically 256); key $K[0], \dots, K[L-1]$ (bytes, where L is the key length).

Ensure: Pseudorandom permutation $S[0], \dots, S[n-1]$.

- ① Initialise: $S[i] \leftarrow i$ for $i = 0, \dots, n-1$ (identity permutation).
- ② Set $j \leftarrow 0$.
- ③ **for** $i = 0$ **to** $n-1$ **do**:
 - $j \leftarrow (j + S[i] + K[i \bmod L]) \bmod n$
 - **Swap**($S[i], S[j]$)
- ④ Reset $i \leftarrow 0, j \leftarrow 0$.

Here $\bmod$ denotes the modulo operation (remainder after integer division). The key byte $K[i \bmod L]$ (where $\bmod$ “wraps around” the key if it is shorter than n) mixes the key throughout the permutation, ensuring that different keys produce different permutations.

2.3.2 Cryptographic PRNGs and RC4 IV

Algorithm 4: RC4 Pseudo-Random Generation Algorithm (PRGA)

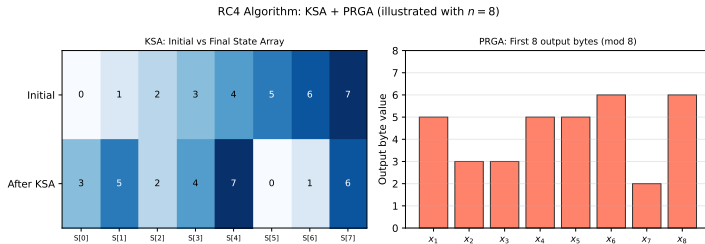
Require: Permutation S from KSA.

Ensure: Output byte stream $x_1, x_2, \dots$

- 1 Set $i \leftarrow 0, j \leftarrow 0, r \leftarrow 0$.
- 2 **while** generating output **do**:
 - $i \leftarrow (i + 1) \bmod n$
 - $j \leftarrow (j + S[i]) \bmod n$
 - **Swap**($S[i], S[j]$)
 - $x_r \leftarrow S[(S[i] + S[j]) \bmod n]$
 - $r \leftarrow r + 1$

Each output byte x_r is looked up from the permutation after two index updates and one swap. The permutation evolves with every byte produced, so the internal state is never static.

2.3.2 Cryptographic PRNGs and RC4 V



The figure above illustrates the KSA (left) and PRGA (right) of RC4. In the KSA, the key byte $K[i \bmod L]$ contributes to the index j at each step, folding the secret key throughout the initial permutation. In the PRGA, both counters i and j advance on every step, ensuring that every position in the permutation eventually participates in producing output.

Why RC4 was eventually broken and deprecated. Despite its elegance, RC4 has serious statistical biases. The first few hundred output bytes are correlated with the key. Fluhrer, Mantin, and Shamir (2001) showed that WEP's use of key-derived initialisation vectors allowed full key recovery from

2.3.2 Cryptographic PRNGs and RC4 VI

captured traffic — just by observing the first bytes of many encrypted messages. The IETF deprecated RC4 in RFC 7465 (2015). Modern protocols use AES in counter mode (AES-CTR) or the ChaCha20 stream cipher, both of which have formal security proofs under standard hardness assumptions.

Key Insight: Statistical Correctness is Not Security

RC4 illustrates that a clever design can produce output that looks random while still being cryptographically weak. For cryptographic use, security must be *provably* reducible to a well-studied hard computational problem. For non-cryptographic Monte Carlo use, PCG64 or MT19937 are entirely sufficient — security is not the concern there.

2.3.2 Python Code: RC4 Implementation

```
def rc4_ksa(n, key):
    """Key-Scheduling Algorithm: initialise permutation from key."""
    S = list(range(n))
    j, L = 0, len(key)
    for i in range(n):
        j = (j + S[i] + key[i % L]) % n
        S[i], S[j] = S[j], S[i]
    return S

def rc4_prnga(S, num_bytes):
    """Pseudo-Random Generation Algorithm: produce keystream bytes."""
    n, i, j = len(S), 0, 0
    output = []
    while len(output) < num_bytes:
        i = (i + 1) % n
        j = (j + S[i]) % n
        S[i], S[j] = S[j], S[i]
        t = (S[i] + S[j]) % n
        output.append(S[t])
    return output

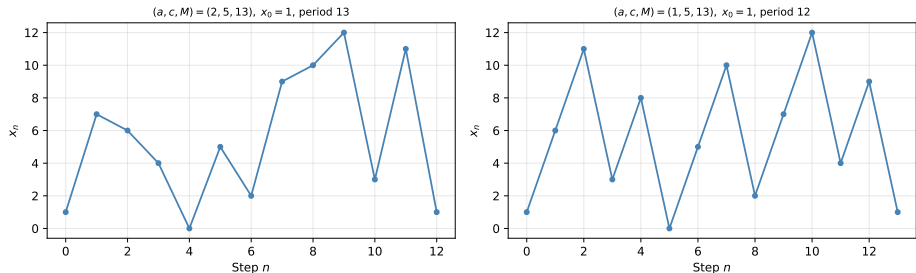
# Example: n=256 (standard), key = [3, 1, 4, 1, 5, 9]
key      = [3, 1, 4, 1, 5, 9]
S        = rc4_ksa(256, key)
stream   = rc4_prnga(S, 20)
print("RC4 keystream (first 20 bytes):", stream)
```

2.4 Simple Generators: Historical Perspective I

Before the development of modern high-quality PRNGs, several simple families of generators were widely used in commercial software and scientific computing. They are now primarily of historical and educational value. They are easy to implement and mathematically transparent, which makes them excellent for understanding the challenges of PRNG design — and for understanding why modern generators are structured the way they are.

The fundamental challenge: lattice structure. Consider the LCG (linear congruential generator) with recurrence $x_j = (137x_{j-1} + 187) \bmod 256$, which has period 256. If we plot all 256 pairs (u_i, u_{i+1}) where $u_i = x_i/256$, the points do not fill the unit square uniformly. Instead they lie on a small number of parallel lines — the *lattice structure* inherent in all LCGs. This structure is immediately visible to the eye and represents a serious failure of two-dimensional uniformity.

2.4 Simple Generators: Historical Perspective II



The figure above shows consecutive pairs from a small LCG (left) and a larger one (right). The parallel-line lattice structure is unmistakable. Any simulation that relies on pairs or tuples of consecutive numbers from such a generator will see artefacts from this structure.

2.4.1 Linear Congruential Generators (LCG) I

The **linear congruential generator (LCG)** is the simplest useful PRNG. It produces a sequence of integers by applying a linear recurrence modulo a large integer. Its appeal is minimal computational cost: one multiplication, one addition, and one modulo operation per step. An enormous amount of theoretical work has been done on LCGs, making them the best-understood generators.

The Linear Congruential Recurrence (Eq. 2.4.1)

$$x_{n+1} = (a x_n + c) \bmod M.$$

Here:

- $M > 0$ is the **modulus** (the size of the state space; outputs x_n lie in $\{0, \dots, M - 1\}$),
- $0 \leq a < M$ is the **multiplier** (controls the step size),
- $0 \leq c < M$ is the **increment** (an additive offset),
- $0 \leq x_0 < M$ is the **seed** (the starting value).

The output is $u_n = x_n/M \in [0, 1)$. When $c = 0$ the generator is called a *linear multiplicative congruential generator (LMCG)*.

2.4.1 Linear Congruential Generators (LCG) II

How to read this formula. At each step, the current state x_n (x subscript n , meaning the n -th state) is multiplied by a , shifted by c , and then reduced modulo M (i.e., we take the remainder when dividing by M , ensuring the result stays in $\{0, \dots, M - 1\}$). A good choice of parameters makes the sequence visit every value in $\{0, \dots, M - 1\}$ before repeating — achieving its maximum possible period of M .

Theorem 2.4.1 — Full-Period Conditions (Knuth)

Let $b = a - 1$. The LCG sequence has period *exactly* M if and only if all three conditions hold simultaneously:

- 1 $\gcd(c, M) = 1$: the increment c is coprime with the modulus (they share no common factors).
- 2 $b = a - 1$ is divisible by every prime p that divides M .
- 3 If 4 divides M , then 4 also divides b , i.e., $a \equiv 1 \pmod{4}$.

Intuition behind the conditions. Condition 1 ensures the increment does not introduce a common factor that would restrict the output to a proper subset of $\{0, \dots, M - 1\}$. Condition 2 ensures the multiplier is “close enough to 1” modulo every prime factor of M that the sequence can reach every

2.4.1 Linear Congruential Generators (LCG) III

residue class. Condition 3 is a technical fix needed when M is divisible by a high power of 2, accounting for how multiplication interacts with factors of 2.

2.4.1 Linear Congruential Generators (LCG) IV

Worked Example 2.4.2: Classical 32-bit LCG

Choose $M = 2^{31} - 1 = 2,147,483,647$ (a Mersenne prime, meaning $2^{31} - 1$ is itself prime), $a = 7^5 = 16,807$, $c = 0$.

Since M is prime, the only divisors are 1 and M itself. With $c = 0$ (LMCG), the period is $M - 1 = 2,147,483,646 \approx 2.1 \times 10^9$. This was among the most widely used generators on 32-bit computers and was proposed by Lewis, Goodman, and Miller. Another popular 32-bit choice is $M = 2^{31} - 1$, $a = 40,014$. For 36-bit machines, $M = 2^{35} - 31$, $a = 5^5$ was recommended.

2.4.1 Linear Congruential Generators (LCG) V

Worked Example 2.4.3: Small LCG (Resing's example)

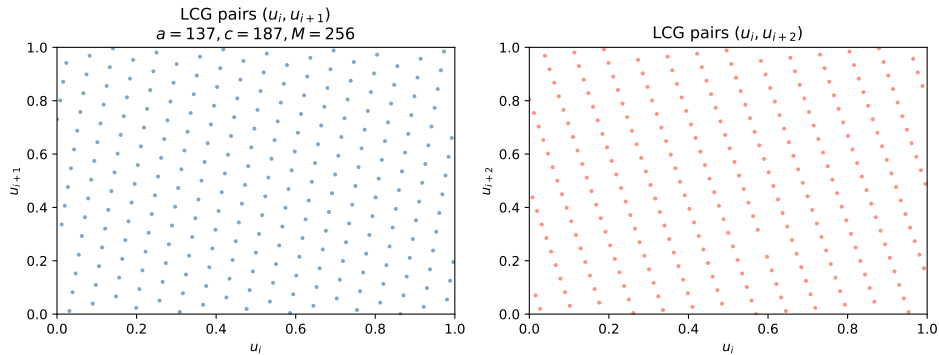
Take $(a, c, M) = (1, 5, 13)$ with seed $x_0 = 1$.

Check Theorem 2.4.1: $b = a - 1 = 0$; $\gcd(5, 13) = 1$ (since 13 is prime and $5 \neq 13$); condition 3 is vacuous since $4 \nmid 13$. All conditions hold. The sequence visits every element of $\{0, 1, \dots, 12\}$ exactly once:

$$1 \rightarrow 6 \rightarrow 11 \rightarrow 3 \rightarrow 8 \rightarrow 0 \rightarrow 5 \rightarrow 10 \rightarrow 2 \rightarrow 7 \rightarrow 12 \rightarrow 4 \rightarrow 9 \rightarrow 1 \rightarrow \dots$$

Period = 13 = M . Now take $(a, c, M) = (2, 5, 13)$ with $x_0 = 1$. Here $b = 1$ and $13 \nmid 1$ (13 does not divide 1), so condition 2 fails. The sequence has period 12, not 13, and starting from $x_0 = 8$ gives the constant sequence $8, 8, 8, \dots$ with period 1.

2.4.1 Linear Congruential Generators (LCG) VI



The figure shows (u_i, u_{i+1}) pairs for a small LCG. The lattice structure is unmistakable: all points lie on a small number of parallel lines rather than filling the square uniformly. This is the content of **Marsaglia's theorem** (1968): the d -dimensional vectors $(u_i, u_{i+1}, \dots, u_{i+d-1})$ produced by any LCG always lie on at most $M^{1/d}$ parallel hyperplanes. Here $M^{1/d}$ denotes M to the power $1/d$ (the d -th root of M). For large M and small d , the hyperplanes may be closely spaced, but they are always present.

2.4.1 Linear Congruential Generators (LCG) VII

Commercial LCG parameters historically used in widely distributed software.

System	Recurrence	Output formula
Java	$x_{i+1} = (25,214,903,917 x_i + 11) \bmod 2^{48}$	$u_i = (2^{27} \lfloor x_{2i}/2^{22} \rfloor + \lfloor x_{2i+1}/2^{21} \rfloor)$
rand48 (UNIX)	same x_i recurrence	$u_i = x_i/2^{48}$
Visual Basic	$x_i = (1,140,671,485 x_{i-1} + 12,820,163) \bmod 2^{24}$	$u_i = x_i/2^{24}$
Excel	$u_i = (0.9821 u_{i-1} + 0.211327) \bmod 1$	direct

All of these generators were chosen for implementation simplicity and speed on the hardware of their era. By modern standards, all of them fail the TestU01 BigCrush test battery and should *not* be used for serious Monte Carlo simulations.

2.4.1 Linear Congruential Generators (LCG) VIII

Generalisations of the linear congruential method.

The **generalised linear congruential generator (GLCG)** extends the recurrence to use multiple previous states:

$$x_n = (a_1x_{n-1} + a_2x_{n-2} + \cdots + a_kx_{n-k} + c) \bmod M.$$

When M is prime and the coefficients a_j are chosen so the characteristic polynomial is a *primitive polynomial* over the Galois field $\text{GF}(M)$, the period can reach M^k — far exceeding the M limit of a simple LCG and making very long periods achievable without an astronomically large modulus.

Quadratic and cubic generators replace the linear recurrence with nonlinear recurrences, which destroy the lattice structure at the cost of slower computation:

- **QCG I**: $x_{i+1} = x_i^2 \bmod M$ where M is a 512-bit prime.
- **QCG II**: $x_{i+1} = 2x_i^2 + 3x_i + 1 \bmod 2^{512}$.
- **CCG (Cubic)**: $x_{i+1} = 2x_i^3 \bmod 2^{512}$.

2.4.1 Linear Congruential Generators (LCG) IX

Fibonacci and Lagged Fibonacci Generators. The Fibonacci generator is $x_n = x_{n-1} + x_{n-2} \bmod M$. The *subtractive lagged Fibonacci generator* (Knuth) uses

$$x_n = x_{n-k} + x_{n-l} \bmod M, \quad n \geq \max(k, l),$$

with recommended parameters $k = 100$, $l = 37$, $M = 2^{30}$. By using two states that are far apart in the sequence (the “lags” k and l), this generator can achieve very long periods with a simple operation.

L'Ecuyer Mixed Generator. To achieve very long periods, L'Ecuyer combines two GLCG sequences. The combined output has a period approximately equal to the *product* of the two component periods, and the lattice structures from the two components partially cancel each other. The generator is defined by:

- ① $x_n = (A_1 x_{n-2} - A_2 x_{n-3}) \bmod M_1$
- ② $y_n = (B_1 y_{n-1} - B_2 y_{n-3}) \bmod M_2$
- ③ $z_n = (x_n - y_n) \bmod M_1$
- ④ **Output:** $u_n = z_n / (M_1 + 1)$ if $z_n > 0$; otherwise $u_n = M_1 / (M_1 + 1)$

2.4.1 Linear Congruential Generators (LCG) X

Parameters: $M_1 = 4,294,967,087$, $M_2 = 4,294,944,443$, $A_1 = 1,403,580$, $A_2 = 810,728$, $B_1 = 527,612$, $B_2 = 1,370,589$. The period is approximately 2^{191} .

Key Insight: Combining Imperfect Generators

Combining two generators with different lattice structures typically yields a better generator than either component alone, because the lattice structures are offset and partially cancel. This “combination” principle underlies all composite PRNGs and is the reason the L’Ecuyer mixed generator achieves a period of $\approx 2^{191}$ while each component has period $\approx 2^{32}$.

2.4.1 Python Code: LCG Implementation and Lattice Visualisation

```
import numpy as np
import matplotlib.pyplot as plt

def lcg(a, c, M, x0, n):
    """Linear Congruential Generator: return n uniform outputs."""
    x, seq = x0, []
    for _ in range(n):
        x = (a * x + c) % M
        seq.append(x / M)
    return seq

# Example 2.4.3: (a,c,M) = (1,5,13), x0=1 => period 13 (full period)
u = lcg(a=1, c=5, M=13, x0=1, n=14)
print("LCG (1,5,13), x0=1:", [f"{v:.3f}" for v in u])

# Example 2.4.3: (a,c,M) = (2,5,13), x0=1 => period 12 (NOT full)
u2 = lcg(a=2, c=5, M=13, x0=1, n=14)
print("LCG (2,5,13), x0=1:", [f"{v:.3f}" for v in u2])

# Visualise lattice structure for a 256-state LCG
u256 = lcg(a=137, c=187, M=256, x0=1, n=256)
plt.figure(figsize=(5, 4))
plt.scatter(u256[:-1], u256[1:], s=4, alpha=0.7)
plt.title("LCG pairs: visible lattice structure")
plt.xlabel(r"$u_i$"); plt.ylabel(r"$u_{i+1}$")
plt.tight_layout(); plt.savefig("lcg_scatter.pdf")
```

2.4.2 Bit Generators: Building Randomness from Binary Operations I

Many PRNGs operate directly on *bits* rather than integers. These **bit generators** produce sequences of $\{0, 1\}$ -valued numbers using only fast bitwise operations (XOR and shifts), making them extremely efficient in hardware. Cryptographic generators also inherently operate at the bit level, since encryption and decryption are fundamentally bitwise.

Fundamental register operations. On modern computers, bit sequences of length n are stored in n -bit registers (typically 32-bit or 64-bit words). Two operations are used in every bit generator:

2.4.2 Bit Generators: Building Randomness from Binary Operations II

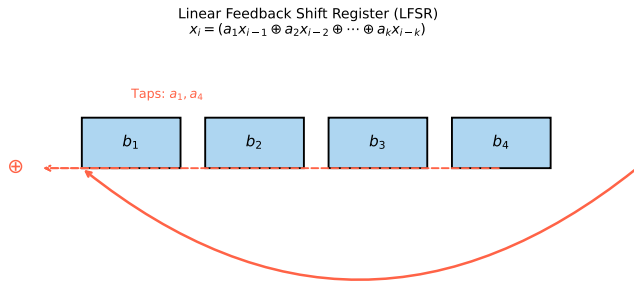
XOR and Bit Shifts

The **exclusive OR** (written $\oplus$, pronounced “XOR”) returns 1 if and only if exactly one of its inputs is 1. For single bits: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$. For bit sequences of length n , XOR is applied element-by-element: $\mathbf{p} \oplus \mathbf{q} = (p_1 \oplus q_1, \dots, p_n \oplus q_n)$. An important property: $x \oplus x = 0$ for all x , and $x \oplus 0 = x$.

The **left shift** $\ll a$ (pronounced “shift left by a ”) moves all bits a positions to the left, filling the a vacated positions on the right with zeros. For example, $x = y \ll 2$ shifts y left by 2 bits, which is equivalent to multiplying by 4 in ordinary arithmetic (but faster).

The **right shift** $\gg b$ is the analogous operation shifting right by b positions, which is equivalent to integer division by 2^b .

2.4.2 Bit Generators: Building Randomness from Binary Operations III



The figure shows the schematic of a Linear Feedback Shift Register (LFSR) with taps at positions $a_1, a_2, \dots, a_k$, which are XORed together to produce the next bit.

2.4.2 Bit Generators: Building Randomness from Binary Operations IV

Exclusive OR Generator (XORG)

The XORG is one of the simplest bitwise generators. It uses the recurrence:

$$x_i = x_{i-1} \oplus x_{i-127}, \quad i \geq 128.$$

The generator is initialised with a 127-bit seed $x_1, \dots, x_{127}$. At each step, the new bit is the XOR of the most recent bit and the bit from 127 steps ago. The “lag” of 127 steps gives the generator some memory, preventing the trivial short cycles of a 1-step recurrence. The XORG is extremely fast but limited in statistical quality.

2.4.2 Bit Generators: Building Randomness from Binary Operations V

Linear Feedback Shift Register (LFSR)

An LFSR uses a k -bit shift register with *feedback taps* to produce the recurrence:

$$x_i = a_1 x_{i-1} \oplus a_2 x_{i-2} \oplus \cdots \oplus a_k x_{i-k},$$

where $a_j \in \{0, 1\}$ are the feedback coefficients (either 0 or 1, determining which past bits contribute to the new bit). The maximum period is $2^k - 1$ (the all-zeros state is absorbing and must be excluded).

When the feedback polynomial $\sum_{j=1}^k a_j x^{j-1}$ is a *primitive polynomial* over $\text{GF}(2)$ (the field with two elements), the LFSR achieves this maximal period.

LFSRs are extremely fast and widely used in hardware (FPGAs, communication chips) because they require only a shift register and a few XOR gates. However, LFSRs are *not* cryptographically secure: given only $2k$ consecutive output bits, one can solve a system of $2k$ linear equations (the Berlekamp-Massey algorithm) to recover all tap coefficients and then predict every future output bit.

2.4.2 Bit Generators: Building Randomness from Binary Operations VI

Blum-Blum-Shub (BBS) Generator — Provably Secure but Slow

The BBS generator is the canonical example of a *provably cryptographically secure* bit generator:

$$x_{i+1} = x_i^2 \bmod M, \quad M = pq,$$

where p and q are large primes both congruent to 3 modulo 4 (i.e., $p \equiv q \equiv 3 \pmod{4}$). The output bit at each step is the least significant bit (the last bit in the binary representation) of x_i .

Security rests on the **quadratic residuosity assumption**: without knowing the factorisation $M = pq$, no polynomial-time algorithm can distinguish quadratic residues (numbers that are perfect squares modulo M) from non-residues with non-negligible probability. This is believed to be as hard as factoring M , which is the basis of RSA encryption.

BBS is provably secure, but computing $x^2 \bmod M$ for a 512-bit number M requires 512-bit modular multiplication, which is about 10^5 times slower than an XOR operation. This makes BBS impractical for high-throughput Monte Carlo use.

2.4.2 Bit Generators: Building Randomness from Binary Operations VII

Shift-XOR Generator (SXR) — Fast and Practical

The SXR generator combines shifts and XOR for a fast, reasonably good non-cryptographic generator operating on an n -bit word:

- 1 Choose seed $x_0 \in \{1, \dots, 2^n - 1\}$ (typically $n = 32$ or 64).
- 2 Define the recurrence: $x_{i+1} = (x_i \oplus (x_i \ll a)) \oplus (x_i \gg b)$, where a and b are chosen shift parameters.
- 3 Output x_{i+1} as the next pseudorandom number.

For $n = 32$ with $a = 13$, $b = 17$: starting from seed $x_0 = 0xABCDE123$, the sequence evolves as $x_1 = (x_0 \oplus (x_0 \ll 13)) \oplus (x_0 \gg 17)$, $x_2 = (x_1 \oplus (x_1 \ll 13)) \oplus (x_1 \gg 17)$, and so on. Properties: very fast (3 operations per step); good statistical quality for simple uses; may fail advanced tests.

2.4.2 Bit Generators: Building Randomness from Binary Operations VIII

Key Insight: The Speed-Security Spectrum

The three generators form a spectrum. LFSR is the fastest (one XOR per bit) but is trivially broken by a linear algebra attack. BBS is provably secure but about 10^5 times slower. SXR is a practical middle ground for non-cryptographic Monte Carlo simulations, trading some security for speed while maintaining good statistical properties.

2.5 Testing Good Properties of PRNGs I

The quality of a PRNG directly determines the reliability of any simulation, integral estimate, or statistical analysis built on it. Statistical testing is how we gain confidence that a generator's output is *indistinguishable* from truly random output for all practical purposes.

There are two complementary approaches to assessing PRNG quality:

Theoretical tests examine the internal structure of the generator. The *lattice test* analyses the geometry of the point set $\{(u_i, u_{i+1}, \dots, u_{i+d-1}) : i = 0, 1, \dots\}$ in $[0, 1)^d$; the *spectral test* measures the minimum distance between parallel hyperplanes containing all these points. A good generator should have large minimum distances (densely spaced hyperplanes) in all dimensions.

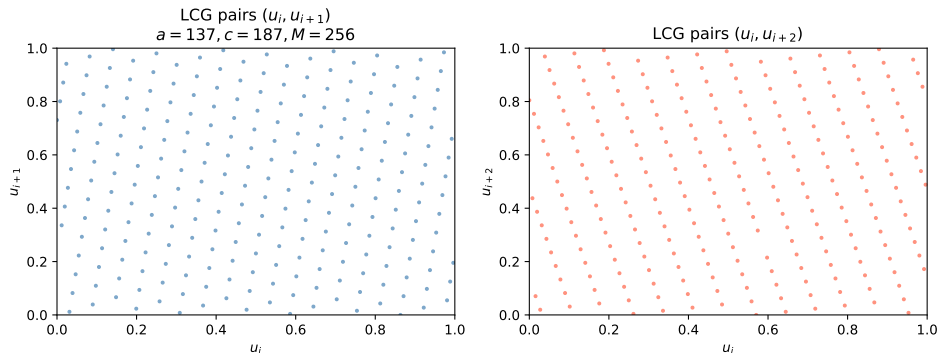
Statistical tests treat the PRNG as a black box and apply hypothesis tests to the output sequence. The question is: could this sequence plausibly have been produced by a truly random source?

Major test batteries in current use:

- **Diehard** (Marsaglia): 18 tests; historically important, now superseded.
- **TestU01** (L'Ecuyer and Simard): SmallCrush (10 tests), Crush (96 tests), BigCrush (106 tests). The most demanding battery in current use; BigCrush requires approximately 10^{10} random numbers.

2.5 Testing Good Properties of PRNGs II

- **NIST SP 800-22:** 15 tests designed for cryptographic PRNGs, emphasising bit-level properties.



The figure shows the motivating example: pairs (u_i, u_{i+1}) from a simple 256-state LCG lie on a few parallel lines, immediately failing a visual test for 2D uniformity. A good generator's pairs would fill the square densely.

2.5.1 Preliminary Remarks: Notation and Conversions I

Before describing specific tests we establish notation and address two practical matters: how to convert between output formats, and how to organise a sequence into multi-dimensional vectors for testing.

Notation. We use capital letters for random variables and lowercase letters for their realisations (observed values):

- $U \sim \mathcal{U}[0, 1)$: uniformly distributed on $[0, 1)$. Realisation: u (a specific observed value of U).
- $B \sim \mathcal{U}(\{0, 1\})$: a uniform random bit. Realisation: b .
- $Y \sim \mathcal{U}(\bar{M})$: uniformly distributed on $\{0, \dots, M - 1\}$. Realisation: y .

2.5.1 Preliminary Remarks: Notation and Conversions II

2.5.1.1 Converting Numbers: The Right Way and the Wrong Way

Given a sequence $u_1, u_2, \dots$ in $[0, 1)$, convert to integers via

$$y_i = \lfloor Mu_i \rfloor. \quad (2.5.1)$$

Here $\lfloor Mu_i \rfloor$ (floor of M times u_i) rounds Mu_i down to the nearest integer, giving a value in $\{0, \dots, M-1\}$. Conversely, $u_i = y_i/M$ recovers a $[0, 1)$ value.

To convert to bits, first convert u_i to $y_i \in \{0, \dots, 2^k - 1\}$ using $M = 2^k$, then extract the binary digits of y_i .

Why the naive threshold $b_i = 1[u_i \geq 0.5]$ is dangerous. Suppose $u_i = (0.b_1b_2 \dots b_{24})_2$ is a 24-bit float from Visual Basic's LCG. The threshold rule keeps only bit b_1 (the most significant bit) and discards $b_2, \dots, b_{24}$. A generator that sets $b_2 = \dots = b_{24} = 0$ for every output (making all odd outputs identically zero) would pass the threshold test perfectly but would fail any proper bit-level test.

2.5.1 Preliminary Remarks: Notation and Conversions III

2.5.1.2 Grouping Numbers into Multi-Dimensional Vectors

Given $n = mr$ numbers $u_1, \dots, u_n$, group them into r vectors of length m (vectors of dimension m):

$$\mathbf{u}_1 = (u_1, \dots, u_m), \quad \mathbf{u}_2 = (u_{m+1}, \dots, u_{2m}), \quad \dots, \quad \mathbf{u}_r = (u_{(r-1)m+1}, \dots, u_{rm}).$$

Each $\mathbf{u}_i \in [0, 1)^m$ is a point in the m -dimensional unit hypercube. After converting via $y_j = \lfloor Mu_j \rfloor$, we obtain r integer vectors $\mathbf{y}_i \in \{0, \dots, M-1\}^m$. Similarly for bits: group $n = mr$ bits into r bit vectors $\mathbf{b}_i \in \{0, 1\}^m$. This multi-dimensional grouping is the framework used in virtually all test batteries, because independence in multiple dimensions is much harder to achieve than independence in one dimension.

2.5.1 Preliminary Remarks: Notation and Conversions IV

2.5.1.3 The p -Value Concept

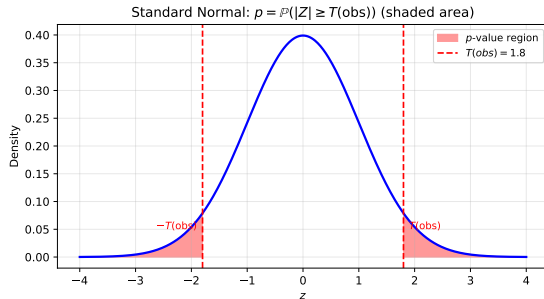
For a test statistic T with cumulative distribution function (c.d.f.) F under the null hypothesis $\mathcal{H}_0$ (“the sequence is truly random”), the p -value is the probability of observing a statistic at least as extreme as the observed value $T(\text{obs})$:

$$p = \mathbb{P}(T \geq T(\text{obs})) = 1 - F(T(\text{obs})).$$

Under $\mathcal{H}_0$ with a continuous statistic T , the p -value is itself uniformly distributed: $p \sim \mathcal{U}[0, 1)$.
Therefore:

- A *small* p (e.g., $p < 0.01$) is evidence *against* $\mathcal{H}_0$: the sequence is unlikely to be random.
- A suspiciously *large* p (e.g., $p > 0.99$) also indicates non-randomness: the sequence is “too regular” to be natural. This is exactly what happens with quasi-random (low-discrepancy) sequences, which are designed to be more uniform than random.

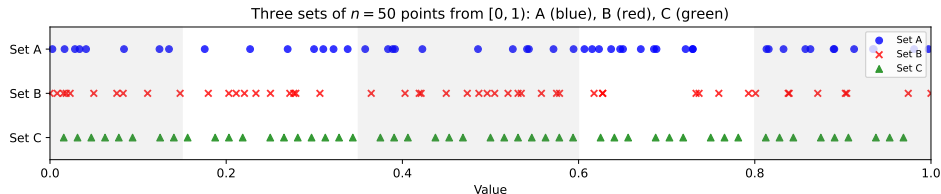
2.5.1 Preliminary Remarks: Notation and Conversions V



2.5.2 Two Fundamental Goodness-of-Fit Tests I

We illustrate the two fundamental tests using three sets of $n = 50$ points from $[0, 1]$:

- **Set A** (blue): generated by PCG64 via NumPy's `default_rng`. This is the pseudorandom baseline.
- **Set B** (red): a biased nonlinear transform of Set A: $u_i^B = (e^{u_i^A - 1} - m)/(M - m)$ where m, M are the min and max. This set is skewed toward 0 and should be rejected.
- **Set C** (green): a quasi-random equispaced sequence. It is “too uniformly distributed” to be random and should be detected by second-level testing (Section 2.5.6).



2.5.2 Two Fundamental Goodness-of-Fit Tests II

2.5.2.1 Kolmogorov-Smirnov (KS) Goodness-of-Fit Test

The KS test compares the **empirical c.d.f.** — the step function that records the fraction of observed values below each threshold:

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(U_i \leq t)$$

— with the theoretical c.d.f. $F(t) = t$ for $\mathcal{U}[0, 1)$. Here $\mathbf{1}(U_i \leq t)$ equals 1 if $U_i \leq t$ and 0 otherwise, so $\hat{F}_n(t)$ simply counts the fraction of sample values that are at most t .

The **KS statistic** D_n is the maximum absolute deviation between the empirical and theoretical c.d.f.s:

$$D_n = \sup_{0 \leq t \leq 1} |\hat{F}_n(t) - t|.$$

Here $\sup$ (supremum) means the maximum over all values of $t \in [0, 1]$. By the Glivenko-Cantelli theorem, $D_n \rightarrow 0$ almost surely when the data are i.i.d. uniform. The normalised statistic $\sqrt{n} D_n$ converges in distribution (as $n \rightarrow \infty$) to the Kolmogorov distribution with c.d.f.

$$K(t) = 1 - 2J(\exp(-2t^2)), \quad t > 0,$$

where $J(t) = \sum_{j=1}^{\infty} (-1)^{j-1} \exp(-2j^2 t^2)$ is a series that can be computed numerically. Critical values

2.5.2 Python Code: KS and Chi-Square Tests

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(seed=0)
n = 50
# Set A: PCG64 pseudorandom
A = rng.random(n)
# Set B: biased nonlinear transform of A
eps = 1e-7
raw = np.exp(A - 1) - 1 - eps
B = (raw - raw.min()) / (raw.max() - raw.min())
# Set C: quasi-random equispaced (approximates Halton sequence)
C = np.arange(1, n + 1) / (n + 1)

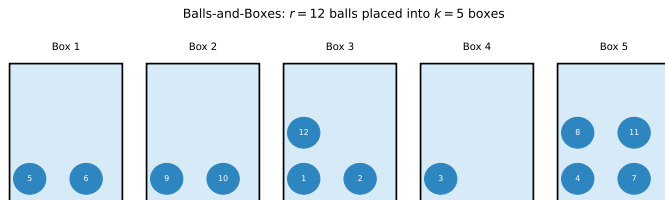
# --- KS test against U[0,1) ---
for name, seq in [("A", A), ("B", B), ("C", C)]:
    stat, p = stats.kstest(seq, 'uniform')
    print(f"KS Set {name}: D={stat:.4f}, p={p:.4f}")

# --- Chi-square test with 5 non-uniform bins from the book ---
bins = [0, 0.15, 0.35, 0.60, 0.80, 1.0]
probs = [0.15, 0.20, 0.25, 0.20, 0.20]
for name, seq in [("A", A), ("B", B), ("C", C)]:
    obs, _ = np.histogram(seq, bins=bins)
    exp = [n * p for p in probs]
    chi2, p_val = stats.chisquare(obs, exp)
    print(f"Chi2 Set {name}: X2={chi2:.3f}, p={p_val:.4f}")
```

2.5.3 Tests Based on Balls-and-Boxes Schemes I

Many classical PRNG tests can be cast as **balls-and-boxes problems**. In the general scheme, we place r balls into k boxes independently and uniformly at random (each ball goes into a uniformly chosen box, independently of all other balls). Under the null hypothesis $\mathcal{H}_0$, the occupancy counts follow a multinomial distribution.

To convert a PRNG output sequence (u_j) into a balls-and-boxes scheme: form r vectors $\mathbf{u}_1, \dots, \mathbf{u}_r$ each of length m , then partition $[0, 1)^m$ into $k = L^m$ sub-hypercubes. Each vector $\mathbf{u}_i$ maps to exactly one sub-hypercube (the “box” it lands in). The choice of m , L , and k defines the specific test.



2.5.3 Tests Based on Balls-and-Boxes Schemes II

The Birthday Problem: when do first collisions appear? Consider r students whose birthdays are uniformly distributed among $k = 365$ days. The probability that all birthdays are distinct is

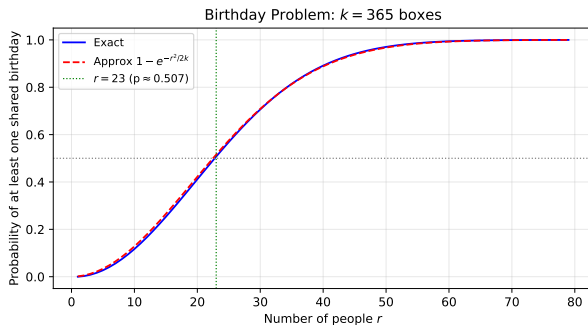
$$\frac{(365)_r}{365^r} = \prod_{j=1}^{r-1} \left(1 - \frac{j}{365}\right),$$

where $(365)_r = 365 \cdot 364 \cdots (365 - r + 1)$ is the falling factorial (the product of r consecutive integers counting down from 365). The probability of at least one shared birthday (a “collision”) is $p = 1 - (365)_r / 365^r$. With $r = 23$ students, $p \approx 0.507$; with $r = 70$, $p \approx 0.999$. For general k boxes and r balls, the collision probability is:

$$p_{k,r} = 1 - \prod_{j=1}^{r-1} \left(1 - \frac{j}{k}\right) \approx 1 - \exp\left(-\frac{r^2}{2k}\right), \quad (2.5.12)$$

where the approximation uses $\log(1 - x) \approx -x$ for small x .

2.5.3 Tests Based on Balls-and-Boxes Schemes III



2.5.3 Tests Based on Balls-and-Boxes Schemes IV

Collision Test

A *collision* occurs when a ball lands in a box that already contains at least one ball. Let $C_{k,r}$ be the number of collisions after placing r balls in k boxes. The exact probability of exactly c collisions:

$$\mathbb{P}(C_{k,r} = c) = \frac{k(k-1) \cdots (k-r+c+1) S_2(r, r-c)}{k^r}, \quad (2.5.13)$$

where $S_2(r, s)$ is the *Stirling number of the second kind*: the number of ways to partition a set of r labeled objects into s non-empty unlabeled subsets. Here $k(k-1) \cdots (k-r+c+1)$ is the number of ways to assign the $r-c$ non-empty groups to distinct boxes.

The **Poisson approximation**: when $r, k \rightarrow \infty$ with $r^2/(2k) \rightarrow \lambda$ (lambda, a fixed positive number):

$$\mathbb{P}(C_{k,r} = c) \rightarrow \frac{\lambda^c}{c!} e^{-\lambda}. \quad (2.5.14)$$

Here $e \approx 2.718$ is Euler's number. This says the number of collisions is approximately Poisson distributed with mean $\lambda = r^2/(2k)$.

2.5.3 Tests Based on Balls-and-Boxes Schemes V

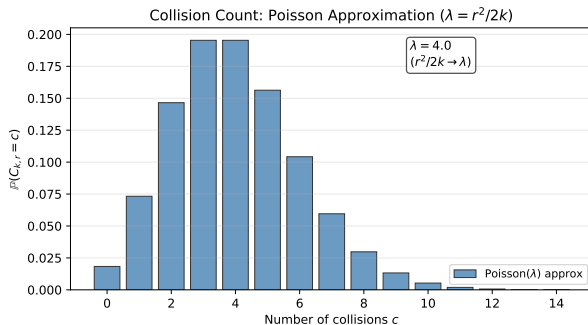
Expected Number of Collisions (Eq. 2.5.17)

The expected (average) number of collisions is:

$$\mathbb{E}C_{k,r} = r - k \left(1 - \left(1 - \frac{1}{k} \right)^r \right) \approx \frac{r^2}{2k}.$$

For example, with $r = 2^{14} = 16,384$ balls and $k = 2^{20} \approx 10^6$ boxes: $\mathbb{E}C_{k,r} \approx 2^{28}/2^{21} = 128$.

2.5.3 Tests Based on Balls-and-Boxes Schemes VI



2.5.3 Tests Based on Balls-and-Boxes Schemes VII

Birthday Spacing Test (Marsaglia). This is among the most stringent tests for LCGs. Given Rn bits, split them into R subsequences each of length $n = mr$ (with $m = 25$, $r = 2^9 = 512$). For each subsequence, treat the r 25-tuples (interpreted as 25-bit integers in $\{0, \dots, 2^{25} - 1\}$) as “birthdays” in a year of $k = 2^{25}$ days. Sort the birthdays and compute the spacings between consecutive sorted values; let K be the number of equal spacings. Under $\mathcal{H}_0$, K approximately follows a Poisson distribution with mean $\lambda = r^3/(4k)$. Apply a chi-square test to the R values of K across all R subsequences. This test fails the MATLAB `rand('state')` LCG with a p -value of 4.12×10^{-100} — a number so small that the generator is conclusively non-random.

Poker Test. Group $n = 5r$ integers $y_1, \dots, y_n \in \{0, \dots, M - 1\}$ into r 5-tuples and classify each by the number of *distinct values* $s \in \{1, 2, 3, 4, 5\}$ (analogous to poker hands). The theoretical probability of a 5-tuple having exactly s distinct values is:

$$p_s = \frac{M(M-1) \cdots (M-s+1) S_2(5, s)}{M^5},$$

where $S_2(5, 1) = 1$, $S_2(5, 2) = 15$, $S_2(5, 3) = 25$, $S_2(5, 4) = 10$, $S_2(5, 5) = 1$. Apply a chi-square test with 4 degrees of freedom.

2.5.3 Tests Based on Balls-and-Boxes Schemes VIII

Key Insight: Balls-and-Boxes Unifies Many Tests

The balls-and-boxes framework unifies the birthday, collision, spacing, and poker tests into a single conceptual framework. The birthday spacing test is especially sensitive to LCGs because their lattice structure causes spacings between consecutive sorted values to be non-uniform.

2.5.4 Tests Based on Random Walks I

Random walk tests convert a bit sequence into a 1D random walk and then test statistical properties of that walk whose distributions are known exactly under $\mathcal{H}_0$. Because the theory of symmetric random walks is rich (Central Limit Theorem, Arcsine Law, excursion theory), these tests achieve precise p -value computation.

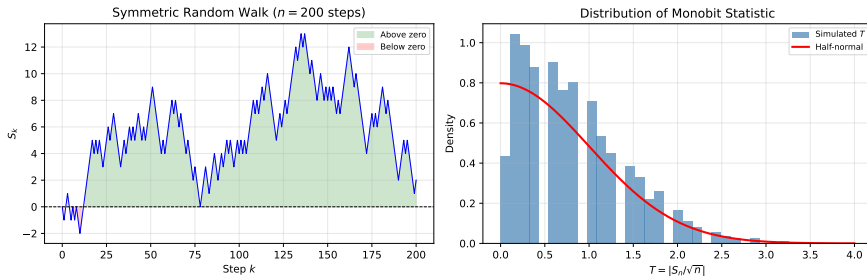
From Bits to a Random Walk (Eq. 2.5.20)

Given bits $b_1, b_2, \dots$, define the centred increments and partial sums:

$$X_i = 2B_i - 1 \in \{-1, +1\}, \quad S_0 = 0, \quad S_k = \sum_{i=1}^k X_i.$$

Here $X_i = +1$ when $b_i = 1$ (a step to the right) and $X_i = -1$ when $b_i = 0$ (a step to the left). Under $\mathcal{H}_0$, each step is equally likely to be $+1$ or -1 , so the walk is *symmetric*: it has no tendency to drift in either direction.

2.5.4 Tests Based on Random Walks II



The figure shows a typical symmetric random walk of length $n = 1000$. The walk drifts up and down with no systematic trend.

2.5.4 Tests Based on Random Walks III

Frequency (Monobit) Test

By the Central Limit Theorem (CLT), the sum $S_n = \sum_{i=1}^n X_i$ satisfies $S_n/\sqrt{n} \xrightarrow{\mathcal{D}} \mathcal{N}(0, 1)$ as $n \rightarrow \infty$, where $\mathcal{N}(0, 1)$ is the standard normal distribution (bell curve with mean 0 and variance 1). The test statistic is $T(\text{obs}) = |S_n|/\sqrt{n}$ (the absolute value of S_n normalised by $\sqrt{n}$), and the p -value is:

$$p = \operatorname{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right),$$

where $\operatorname{erfc}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt$ is the complementary error function (a standard mathematical function giving the tail probability of a Gaussian).

The monobit test checks whether the number of 0s and 1s in the bit sequence is approximately equal. A biased bit stream (more 1s than 0s, or vice versa) will produce a large $|S_n|$, giving a small p -value and rejecting the hypothesis of randomness.

2.5.4 Tests Based on Random Walks IV

Frequency Test in Blocks

Split $n = mr$ bits into r blocks of length m (m bits per block). For block i , let $S_m^{(i)}$ be the walk position at the end of that block. The normalised walk ends $Z_i = S_m^{(i)} / \sqrt{m}$ are approximately i.i.d. $\mathcal{N}(0, 1)$ for large m . The test statistic is:

$$T(\text{obs}) = \frac{1}{m} \sum_{i=1}^r (S_m^{(i)})^2 \approx \chi_r^2.$$

This tests whether the balance of 0s and 1s holds *within every block*, not just globally. A generator with a long-range trend in bit frequencies would pass the monobit test but fail the block frequency test.

2.5.4 Tests Based on Random Walks V

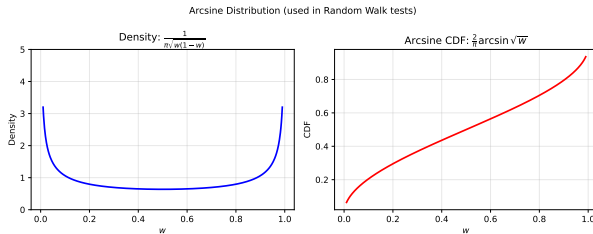
Arcsine Law Based Test

A remarkable and counter-intuitive property of the symmetric random walk is the **Arcsine Law** (Feller, 1968): the fraction of time the walk spends above zero does *not* concentrate near $1/2$. Instead, defining L_{2r}^+ as the number of steps during which $S_k > 0$ (or $S_{k-1} > 0$):

$$\lim_{r \rightarrow \infty} \mathbb{P}\left(\frac{L_{2r}^+}{2r} \leq w\right) = \frac{2}{\pi} \arcsin \sqrt{w}, \quad w \in [0, 1].$$

Here $\arcsin$ (arcsine) is the inverse sine function, and π (pi) ≈ 3.14159 . The density $\frac{1}{\pi\sqrt{w(1-w)}}$ is U-shaped: it is highest near 0 and 1, meaning the walk is *most likely* to spend almost all its time above zero, or almost all its time below zero. Spending exactly half the time above zero has probability zero. This counterintuitive result forms the basis of a test: if a sequence is truly random, the fraction $T(\text{obs}) = L_{2r}^+/(2r)$ should follow the arcsine distribution, giving $p \approx 1 - \frac{2}{\pi} \arcsin \sqrt{T(\text{obs})}$.

2.5.4 Tests Based on Random Walks VI



2.5.4 Tests Based on Random Walks VII

Random Excursion Test (NIST)

An *excursion* is a maximal segment of the walk between two consecutive returns to zero (a segment that starts and ends at $S_k = 0$ without touching zero in between). Let J_n denote the number of times $S_k = 0$ for $k = 1, \dots, n$ (the number of returns to zero). Under $\mathcal{H}_0$:

$$\lim_{n \rightarrow \infty} \mathbb{P}\left(\frac{J_n}{\sqrt{n}} \leq t\right) = \sqrt{\frac{2}{\pi}} \int_0^t e^{-x^2/2} dx.$$

NIST recommends rejecting $\mathcal{H}_0$ if $J_n(\text{obs}) \leq 0.008\sqrt{n}$ (too few returns to zero).

For a value $x \in \{1, 2, \dots, 7\}$ (representing seven specific states of interest), let $L(x)$ be the number of visits to state x during one excursion. The distribution of $L(x)$ under $\mathcal{H}_0$ is:

$$\mathbb{P}(L(x) = k) = \begin{cases} 1 - \frac{1}{2|x|}, & k = 0, \\ \frac{1}{4x^2} \left(1 - \frac{1}{2|x|}\right)^{k-1}, & k \geq 1. \end{cases} \quad (2.5.22)$$

Here $|x|$ denotes the absolute value of x . For each x , count observed visits $E_k(x)$ for $k = 0, 1, \dots, 4$ (assuming $k \geq 5$ visits are extremely rare) as well as $E_{\geq 5}(x)$, the number of excursions with 5 or more visits to state x .

2.5.5 Permutation Tests I

Permutation tests convert blocks of pseudorandom numbers into permutations and test statistical properties of those permutations whose distributions are known exactly under $\mathcal{H}_0$. They exploit the rich combinatorial theory of permutations (derangements, fixed points) to build precise tests.

Derangements (Lemma 2.5.10)

A **derangement** of $\{1, \dots, m\}$ is a permutation σ with *no fixed points*: $\sigma(i) \neq i$ for all $i = 1, \dots, m$. (In other words, no element stays in its original position.) The number of derangements of a set of m elements is:

$$d_m = m! \sum_{k=0}^m \frac{(-1)^k}{k!} = m! \left(1 - 1 + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^m}{m!} \right).$$

Here $k!$ (k factorial) is the product $1 \times 2 \times \dots \times k$, and $(-1)^k$ alternates between $+1$ and -1 . As $m \rightarrow \infty$, $d_m/m! \rightarrow e^{-1} \approx 0.3679$. This means approximately 36.79% of all random permutations are derangements.

2.5.5 Permutation Tests II

Worked example for $m = 4$. $d_4 = 4! \left(1 - 1 + \frac{1}{2} - \frac{1}{6} + \frac{1}{24}\right) = 24 \times \frac{9}{24} = 9$. There are 9 derangements of $\{1, 2, 3, 4\}$ out of $4! = 24$ total permutations. The probability of a derangement is $9/24 = 0.375 \approx e^{-1} = 0.3679$.

2.5.5 Permutation Tests III

Derangement Permutation Test

Require: sequence $u = (u_1, \dots, u_n)$ with $n = mr$.

- ① Initialise: $\xi \leftarrow 0$.
- ② **for** $i = 1$ to r : generate permutation σ_i from the i -th m -tuple $(u_{(i-1)m+1}, \dots, u_{im})$ via Fisher-Yates; if σ_i is a derangement, increment ξ .
- ③ Under $\mathcal{H}_0$: the fraction $\xi/r \approx e^{-1}$, so the normalised test statistic is:

$$T(\text{obs}) = \frac{|\xi - re^{-1}|}{\sqrt{re^{-1}(1 - e^{-1})}}, \quad p = \text{erfc}\left(\frac{T(\text{obs})}{\sqrt{2}}\right).$$

Here $e^{-1} \approx 0.3679$ is the theoretical fraction of derangements, re^{-1} is the expected count of derangements in r permutations, and the denominator is the standard deviation of the count under $\mathcal{H}_0$.

2.5.5 Permutation Tests IV

Fixed-Point Permutation Test

For each m -tuple, generate permutation σ via Fisher-Yates and count the fixed points $\xi = \#\{j : \sigma(j) = j\}$ (the number of elements that stay in their original position).

Under $\mathcal{H}_0$: $\xi \xrightarrow{\mathcal{D}} \text{Poisson}(1)$ (Poisson distribution with mean 1) as $m \rightarrow \infty$. Group observed fixed-point counts into 11 boxes $A_j = \{j\}$ for $j = 0, \dots, 9$ and $A_{10} = \{10, 11, \dots\}$, with expected probabilities $p_j = e^{-1}/j!$ for $j \leq 9$ and $p_{10} = 1 - \sum_{j=0}^9 p_j$. Apply a chi-square test with 10 degrees of freedom.

2.5.5 Python Code: Permutation Tests

```
import numpy as np
from scipy import stats

def fisher_yates(m, u):
    """Generate permutation of {0,...,m-1} from m uniform inputs."""
    S = list(range(m))
    for i in range(m, 0, -1):
        k = int(i * u[i - 1])      # uniform in {0,...,i-1}
        S[k], S[i - 1] = S[i - 1], S[k]
    return S

def is_derangement(sigma):
    return all(sigma[i] != i for i in range(len(sigma)))

rng = np.random.default_rng(42)
m, r = 10, 2000                # 10-tuples, 2000 repetitions
n = m * r
u = rng.random(n)

# ---- Derangement test ----
xi = sum(is_derangement(fisher_yates(m, u[i * m:(i + 1) * m]))
         for i in range(r))
e_inv = np.exp(-1)
T_obs = abs(xi - r * e_inv) / np.sqrt(r * e_inv * (1 - e_inv))
p_val = stats.erfc(T_obs / np.sqrt(2))
print(f"Derangements: {xi}/{r}  T={T_obs:.3f}  p={p_val:.4f}")
```

2.5.6 p -Values and Second-Level Testing I

A single statistical test applied to a single sequence is often not decisive. A genuinely good PRNG might occasionally produce a small p -value by chance (a “false positive”, or Type I error), and a defective generator might pass a particular test for particular parameter choices. **Second-level testing** addresses this by applying the test to *many* independent sequences and checking that the resulting p -values are themselves uniformly distributed.

Why p -Values Are Uniform Under $\mathcal{H}_0$

Suppose T has a continuous, strictly increasing c.d.f. F under $\mathcal{H}_0$. The p -value is $p = 1 - F(T)$. Then:

$$\mathbb{P}(p \leq t) = \mathbb{P}(1 - F(T) \leq t) = \mathbb{P}(F(T) \geq 1 - t) = \mathbb{P}(T \geq F^{-1}(1 - t)) = 1 - F(F^{-1}(1 - t)) = t.$$

So $p \sim \mathcal{U}[0, 1)$ under $\mathcal{H}_0$. This is the key theoretical fact: if the null hypothesis is true, p -values are uniformly distributed. Any deviation from uniformity in a collection of p -values signals non-randomness: values clustering near 0 indicate bias or correlations, while values clustering near 1 indicate “too-uniform” quasi-randomness.

2.5.6 p -Values and Second-Level Testing II

Second-Level Testing Procedure (Eq. 2.5.24–2.5.25):

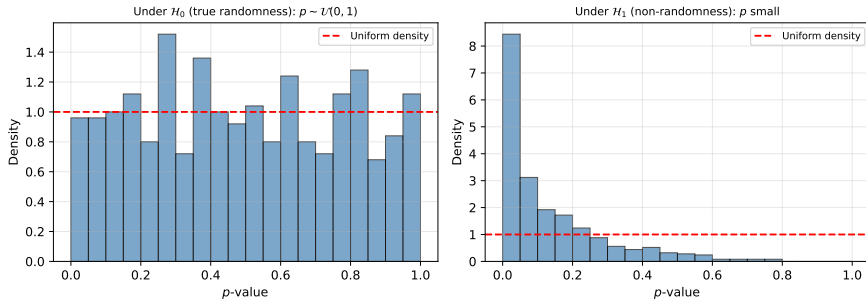
- 1 Split a long sequence of length Rn into R non-overlapping subsequences, each of length n .
- 2 For each subsequence i : compute test statistic $T^{(i)}$ and $p^{(i)} = 1 - F(T^{(i)})$.
- 3 Test whether $p^{(1)}, \dots, p^{(R)}$ are $\mathcal{U}[0, 1)$ by partitioning $[0, 1)$ into s equal bins $[(j-1)/s, j/s)$ for $j = 1, \dots, s$, counting $O_j = |\{i : p^{(i)} \in \text{bin } j\}|$ with expected $E_j = R/s$, and computing:

$$X_{\text{final}}^2 = \sum_{j=1}^s \frac{(O_j - R/s)^2}{R/s} \approx \chi_{s-1}^2.$$

- 4 Compute the final p -value: $p_{\chi^2, \text{final}} = 1 - F_{\chi_{s-1}^2}(X_{\text{final}}^2)$.

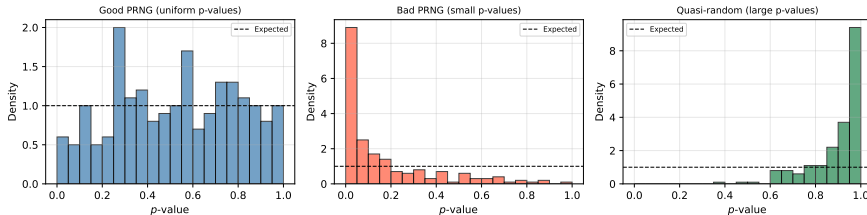
2.5.6 p -Values and Second-Level Testing III

p -value Distributions



2.5.6 p -Values and Second-Level Testing IV

Second-Level Testing: Histograms of $R = 200$ p -values



2.5.6 p -Values and Second-Level Testing V

Practical recommendations (NIST SP 800-22). Use $1000 \leq R \leq 10000$ subsequences, each of length $2^{20} \leq n \leq 2^{32}$. Use significance level $\alpha \in [0.001, 0.01]$. The fraction of sequences passing (having $p^{(i)} > \alpha$) should lie within three standard deviations of $1 - \alpha$:

$$\hat{B} = \frac{1}{R} \sum_{i=1}^R \mathbf{1}[p^{(i)} > \alpha] \in \left[1 - \alpha \pm 3\sqrt{\frac{\alpha(1 - \alpha)}{R}} \right].$$

Here $\mathbf{1}[p^{(i)} > \alpha]$ equals 1 if the i -th subsequence passes the test and 0 otherwise, so $\hat{B}$ is the empirical pass rate.

2.5.6 p -Values and Second-Level Testing VI

Example 2.5.12: Second-Level Test on Sets A, B, C ($R = 200$)

Apply the KS test to $R = 200$ length- $n = 50$ subsequences for each set:

- **Set A** (PCG64): p -values are approximately uniform across $[0, 1)$ — passes the second-level test.
- **Set B** (biased): p -values cluster near 0 because each subsequence exhibits the same leftward skew — *fails*.
- **Set C** (quasi-random): p -values cluster near 1 because each subsequence is “too uniform” — *detected as non-random*, even though first-level KS did not reject it.

Key Insight: Second-Level Testing is Essential

Second-level testing is strictly more powerful than first-level testing. It is the *only* reliable method to detect quasi-random sequences that fool first-level tests by being “too uniform”. It is also the primary defence against overfitting: a generator that was tuned to pass one particular test will still fail the second-level test if its p -values are not uniformly distributed across many independent trials.

2.5.6 Python Code: Second-Level Testing

```
import numpy as np
from scipy import stats

def second_level_test(sequences, test_fn, s=10):
    """
    Run test_fn on each sequence to get a p-value, then test whether
    those p-values are U[0,1) using a chi-square uniformity test.
    sequences : list of 1D arrays
    test_fn    : callable returning a p-value for one sequence
    s          : number of equal bins for the chi-square check
    """
    p_vals = [test_fn(seq) for seq in sequences]
    bins = np.linspace(0, 1, s + 1)
    obs, _ = np.histogram(p_vals, bins=bins)
    exp = [len(p_vals) / s] * s
    chi2, p_final = stats.chisquare(obs, exp)
    return np.array(p_vals), chi2, p_final

# --- Set A: PCG64 pseudorandom sequences -----
rng = np.random.default_rng(0)
R, n = 200, 50
seqs_A = [rng.random(n) for _ in range(R)]

# --- Set C: quasi-random shifted equispaced sequences -----
seqs_C = [(np.arange(n) / n + i / R) % 1.0 for i in range(R)]

def ks_pval(seq):
```

2.6 Bibliographical Notes I

This chapter draws on half a century of PRNG research. The following references provide essential deeper reading.

Foundational references:

- **Knuth** [107] *The Art of Computer Programming, Vol. 2*: the definitive mathematical treatment of LCGs, the full-period theorem (Theorem 2.4.1), the spectral test, Stirling numbers, the serial test, and the poker test. Essential reading for anyone designing or analysing PRNGs.
- **L'Ecuyer** [118, 122]: designed the combined generators of Section 2.4.1 and the TestU01 test suite. His TestU01 paper remains the benchmark reference for PRNG testing; BigCrush comprises 106 tests and requires $\approx 10^{10}$ random numbers to complete.
- **Marsaglia** [141, 145]: invented the Diehard test battery and the birthday spacing test; his 1968 paper proved the lattice structure of all LCGs (the result now known as Marsaglia's theorem).
- **NIST SP 800-22** [187]: the US government statistical test suite for cryptographic PRNGs; 15 standardised tests including the monobit test, block frequency test, and random excursion test.

2.6 Bibliographical Notes II

Modern generators:

- **PCG64** (O'Neill [162]): Permuted Congruential Generators; the current NumPy default via `default_rng`. Uses a 128-bit LCG with a nonlinear output permutation (the PCG “permutation”) that eliminates lattice structure while maintaining a period of 2^{128} .
- **MT19937** (Matsumoto and Nishimura [146]): Mersenne Twister; period $2^{19937} - 1$; passes Diehard but fails some TestU01 BigCrush tests; state size of 624 32-bit integers makes initialisation slow; not suitable for cryptographic use.
- **AES-CTR, ChaCha20**: cryptographically secure stream ciphers replacing RC4 in modern protocols (RFC 7539, RFC 8439). Both have formal security proofs and are the recommended choices for any application where the PRNG output must be computationally unpredictable.

Shuffling and card mixing:

- **Bayer and Diaconis** [14]: proved the “7 shuffle theorem” for the riffle shuffle on 52 cards using Fourier analysis on $\mathcal{S}_{52}$. This paper introduced the cut-off phenomenon for card shuffling.
- **Aldous and Diaconis** [2]: Markov chain coupling methods for mixing-time analysis; provides the theoretical foundation for Section 2.2.2.

2.6 Bibliographical Notes III

Statistical tests:

- **Weiss** [210]: CLT for the empty-boxes count (Proposition 2.5.9).
- **Holst** [83] and **L'Ecuyer et al.** [122]: known results on the serial test chi-square statistic.
- **Tsang et al.** [205]: analysis of the random excursion test.
- **L'Ecuyer and Simard** [169, 170]: design of reliable tests using Berry-Esseen bounds to control normal-approximation errors.

2.7 Selected Exercises I

The following exercises reinforce the theoretical foundations of this chapter.

2.T.1 (Markov and Chebyshev inequalities). For a non-negative random variable X and any constant $a > 0$:

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}X}{a} \quad (\text{Markov inequality}).$$

Here $\mathbb{E}X$ denotes the expected value (average) of X . For a real-valued X with finite variance and any constant $b > 0$:

$$\mathbb{P}(|X - \mathbb{E}X| \geq b) \leq \frac{\text{Var}X}{b^2} \quad (\text{Chebyshev inequality}),$$

where $\text{Var}X = \mathbb{E}[(X - \mathbb{E}X)^2]$ is the variance of X . Chebyshev follows from Markov by applying Markov to $(X - \mathbb{E}X)^2$.

2.T.4. Let $U_1, U_2, \dots$ be i.i.d. $\mathcal{U}[0, 1)$ and $M \geq 2$ an integer. Show that $Y_i = \lfloor MU_i \rfloor$ is a random sequence from $\bar{M} = \{0, \dots, M-1\}$, i.e., the Y_i are i.i.d. with $Y_i \sim \mathcal{U}(\bar{M})$. (Hint: $\mathbb{P}(Y_i = k) = \mathbb{P}(k/M \leq U_i < (k+1)/M) = 1/M$.)

2.7 Selected Exercises II

2.T.8. Verify that Fisher-Yates (Algorithm 1) produces a uniformly random permutation of $\{1, \dots, n\}$. (Hint: by induction on the swap steps, show that after step i , the element in position i is uniformly distributed over the i elements not yet placed in higher positions.)

2.T.17 (Serial test mean). Let O_i ($i = 1, \dots, k$) be the number of random vectors $\mathbf{u}_j$ falling into box i in the serial test. Show that under $\mathcal{H}_0$: $\mathbb{E}X_k^2 = k - 1$.

2.7 Selected Exercises III

2.T.19 (Riffle Shuffle and Time Reversal). Let $p_{RS}(\sigma, \sigma')$ denote the transition probability of a single Riffle Shuffle step, and $p_{TR-RS}(\sigma, \sigma')$ the time-reversed Riffle Shuffle. Show that for any $\sigma, \sigma' \in \mathcal{S}_n$: $p_{RS}(\sigma, \sigma') = p_{TR-RS}(\sigma', \sigma)$.

Lab Exercises:

2.L.1. Implement the LCG with $M = 2^{11} - 1$, $a = 5$, $c = 1$. Generate 10^4 numbers and plot (i, u_i) . Find the period computationally and verify Theorem 2.4.1.

2.L.2. Compare Fisher-Yates (Algorithm 1) to a “broken” version where line 3 is replaced by $k = \text{random}(1, \dots, n)$ (drawing from the full range, not $\{1, \dots, i\}$). Test with the fixed-point permutation test: the broken version should fail because it does not produce a uniform permutation (some arrangements are over-represented).

2.L.3. Generate $n = 10^5$ numbers each with MT19937 and the LMCg ($M = 2^{31} - 1$, $a = 16807$). Compute the Pearson autocorrelation between $(y_1, \dots, y_{n-1})$ and $(y_2, \dots, y_n)$. Which generator shows less autocorrelation?

2.L.5. Quasi-random test: let $u_i = (i \cdot \phi) \bmod 1$ where $\phi = (\sqrt{5} - 1)/2 \approx 0.618$ is the golden ratio (a Kronecker low-discrepancy sequence). For $n = 10^6$ numbers, apply the KS, chi-square, frequency pairs,

2.7 Selected Exercises IV

and second-level KS tests. Observe that first-level tests pass while second-level testing detects the quasi-randomness.

Chapter 2: Summary

Generators:

- PRNG is a 5-tuple (S, s_0, f, V, g) ; period $\leq |S|$.
- Three equivalent sequence types: $\mathcal{U}[0, 1)$, bits, integers.
- Fisher-Yates shuffle: uniform permutation in $\mathcal{O}(n)$.
- 7 riffle shuffles suffice for 52 cards.
- Python: `default_rng/PCG64` (period 2^{128}) and `MT19937` (period $2^{19937} - 1$).
- RC4: KSA+PRGA permutation cipher; deprecated in 2015.
- LCG full-period theorem (Theorem 2.4.1).
- Bit generators: LFSR, XORG, SXR, BBS.
- Combined generators (L'Ecuyer): period $\approx 2^{191}$.

Statistical Testing:

- KS test: $D_n = \sup |\hat{F}_n - t|$.
- Chi-square: $X^2 = \sum (O_i - E_i)^2 / E_i$, need $E_i \geq 5$.
- Serial (pairs): detects 2D correlations.
- Balls-and-boxes: birthday, collision, spacing, poker.
- Random walk: monobit, blocks, arcsine law, excursion.
- Permutation: derangement, fixed-point count.
- Second-level: p -values must be $\mathcal{U}[0, 1)$ across $R \geq 1000$ sequences; detects quasi-randomness.